`/grill-me → /opsx:propose → /opsx:apply → review → /opsx:archive`

Math API: from idea to v1

A Spring Boot REST API built with Claude Code in one afternoon, using grill-me, OpenSpec and ponytail review.

7 November 2026 · github.com/prule/math-api

One afternoon, by the numbers

2 h 47 m

first prompt to last merge; about 1 h 41 m of it active

6 PRs

each merged only after CI passed

6 changes

proposed, applied and archived with OpenSpec

114 tests

unit, web slice, ArchUnit and smoke, written test-first

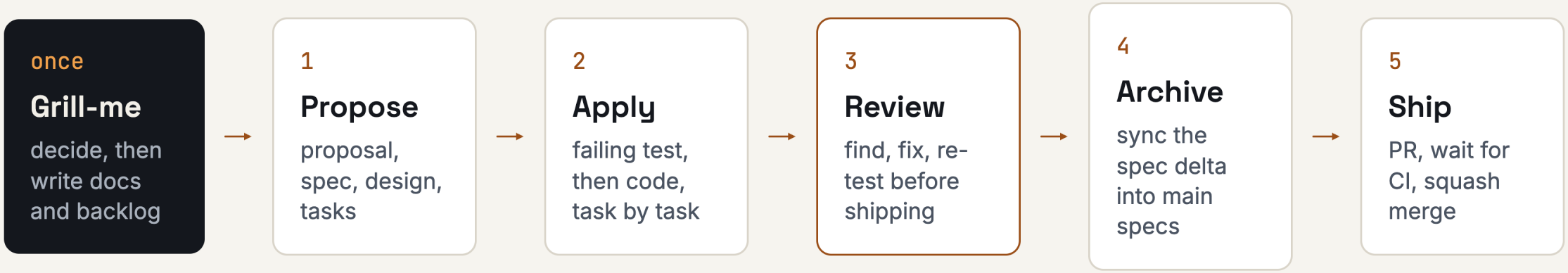
28 / 58

spec requirements / scenarios across 3 capabilities

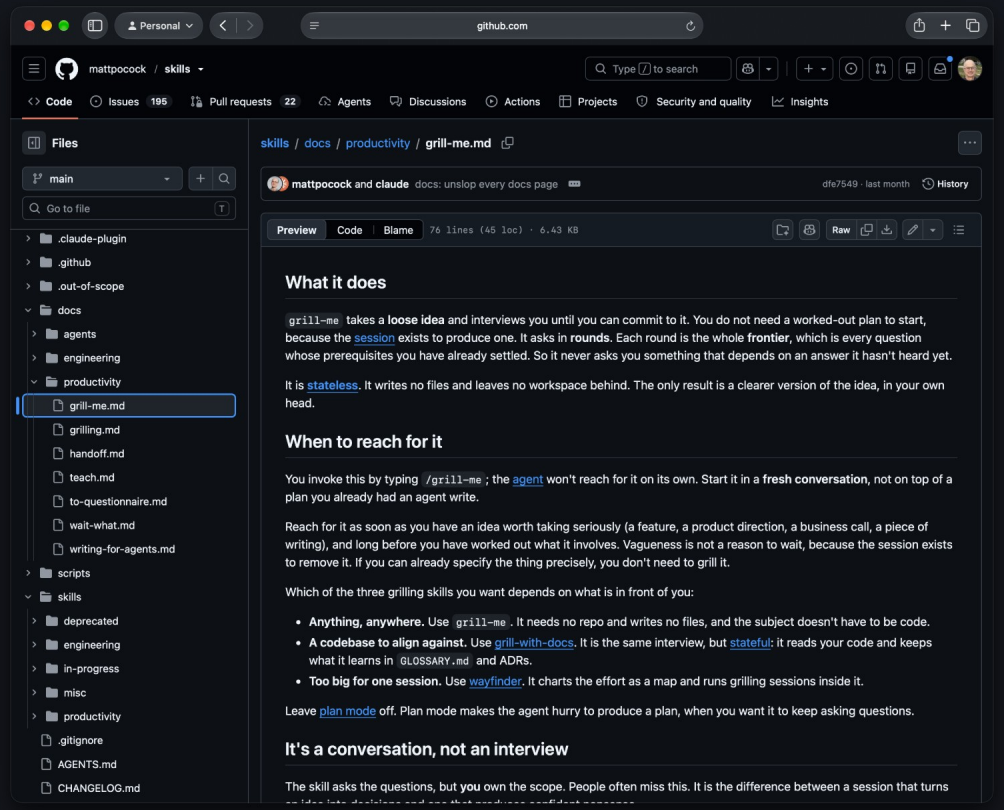
4 docs

PRD, domain model, standards, backlog — updated in every change

The loop we ran for every backlog item



Steps 1–5 repeat per item. The human makes the calls between steps — **“Review first”**, **“Fix, then ship”**, **“Check and merge”** — and nothing merges until CI is green.



/grill-me · 6.5 minutes

A one-line idea, grilled into decisions

- 3 rounds, 24 numbered questions
- Each with a recommended answer; “yes” accepts
- Asked only what the last answers unblocked
- 2 overrides: Java 25 + Boot 4, plain OpenSpec

Four living docs, written once and kept current

PRD.md

Scope, API contract, error table, non-goals and roadmap

DOMAIN-MODEL.md

Glossary (Operand, Operation, Calculation, Result) and business rules BR-1 to BR-4

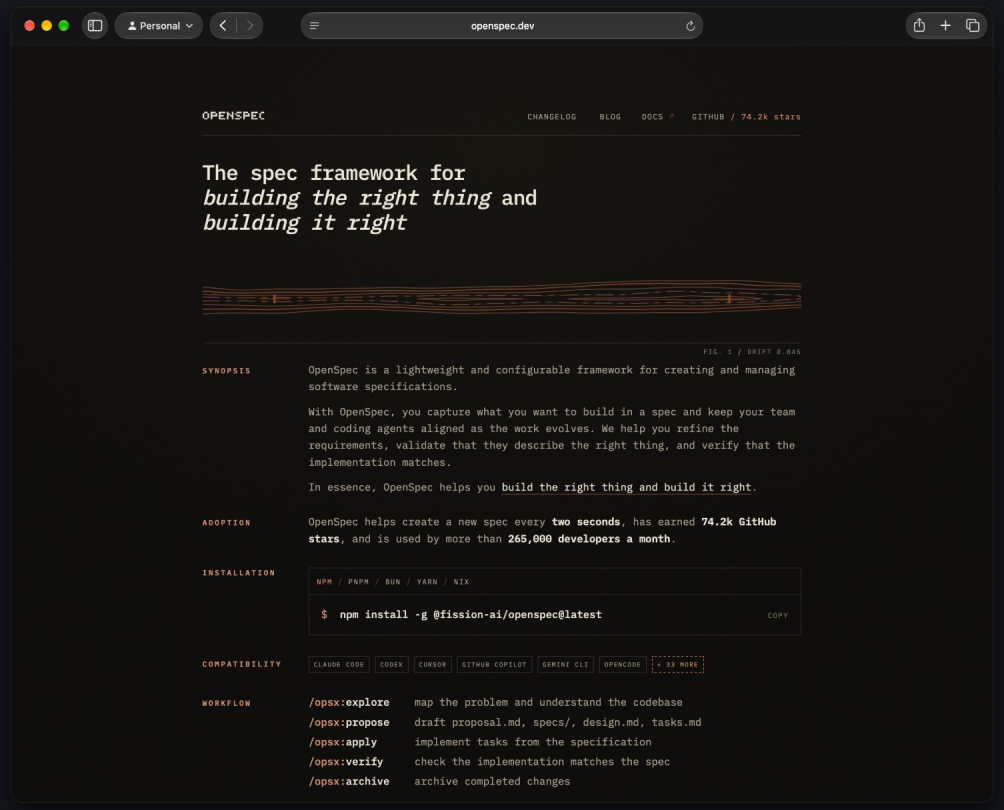
STANDARDS.md

Stack, clean-architecture rules enforced by ArchUnit, testing, workflow, profiles

BACKLOG.md

11 items, each a paste-ready /opsx:propose prompt

Kept current by a CLAUDE.md rule · OpenSpec config rules for proposals and tasks · a PR-template checkbox



OpenSpec 1.14 · spec-driven schema

Plan the change, then build exactly that

/opsx:propose — proposal, spec delta, design, tasks; strict validation

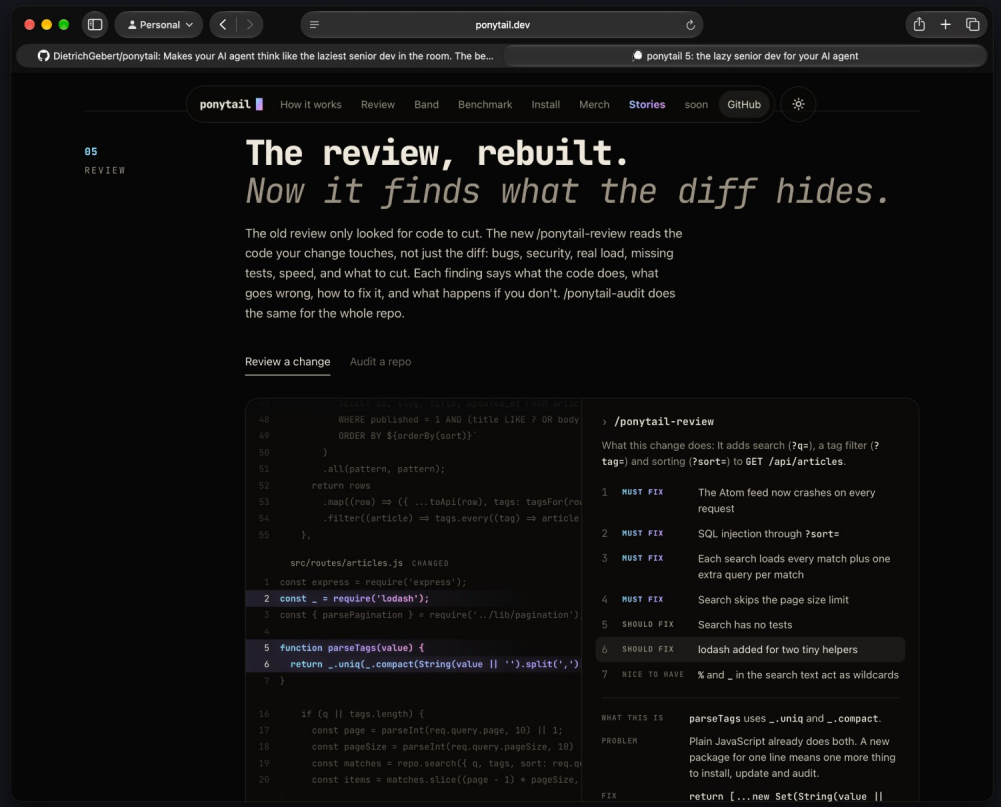
/opsx:apply — tasks in order, failing test first, ticked as they pass

/opsx:archive — sync the delta into main specs, move to archive

Archive with sync grew the specs, change by change

Change	Capability	Delta	Reqs after
add-walking-skeleton	service-health	new, +4	4
add-exact-arithmetic	calculation	new, +10	10
add-divide	calculation	+2	12
add-operand-limits	calculation	+5	17
add-api-docs	api-documentation	new, +6	6
add-api-docs-toggle	api-documentation	2 modified, +1	7

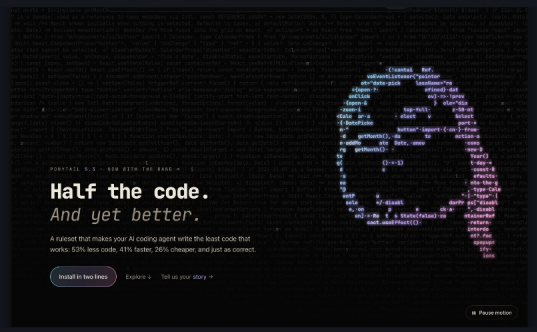
Every sync was checked with openspec validate --strict before the change folder moved to archive/. The first sync caught a spec sentence the running app did not match, and fixed the wording.



ponytail 5.3 · /ponytail:ponytail-review

A review gate before every ship

- “Review first” on items 3, 4 and 5
- Item 5 reviewed twice: the plan, then the code
- ponytail-review on the last change
- Ponytail mode on all session: smallest complete change



What the reviews caught

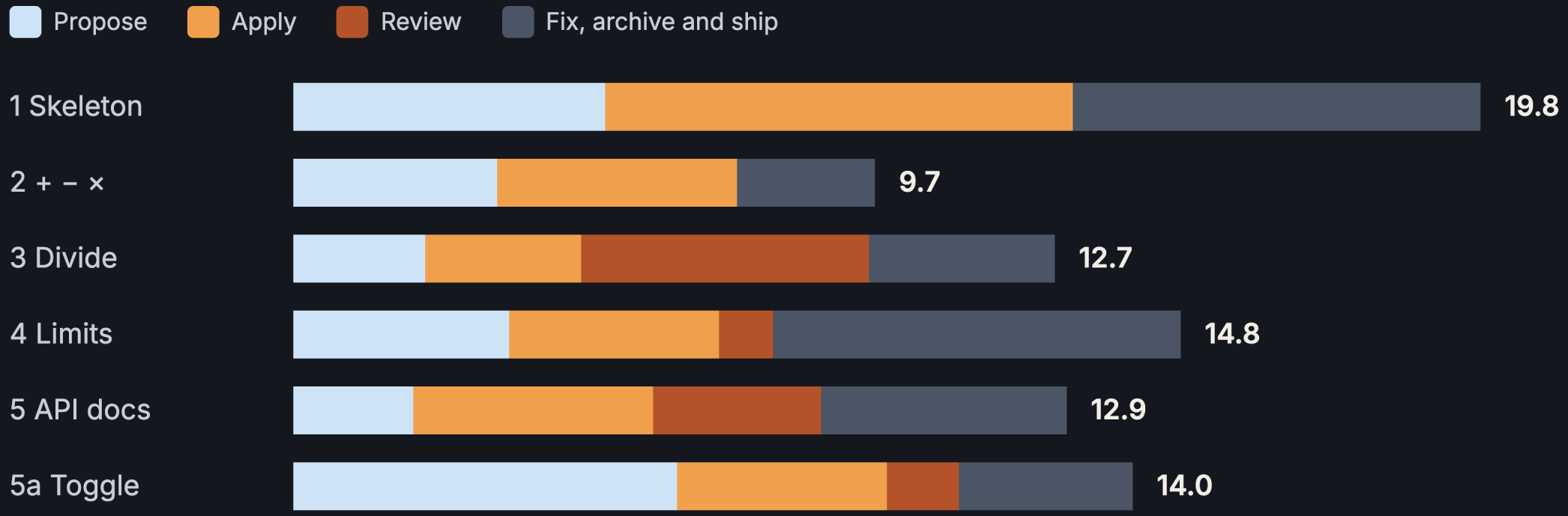
Review	Found	Fixed with
Divide · code	“Unknown operation” test used names we plan to add; 0 ÷ 0 untested through the API	Never-valid name; one more 422 case
Limits · code	Scale check safe only via a (long) cast: 1E+2147483648 has scale Integer.MIN_VALUE	Comment, plus a test that fails without the cast
API docs · plan	200 response would be documented as */*; generic responses could leak in	Explicit 200; exact-codes test
API docs · code	Error schema showed a “properties” field responses never contain	Removed in the customiser; schema pinned
Toggle · apply	Spring still served the Swagger UI webjar under prod	add-mappings: false; 404 test
Toggle · ponytail	Nothing: “Looks good. Ship.”	—

Backlog delivered: v1 plus one follow-up

#	Item	Review	PR	Propose → merge
1	Walking skeleton	—	#1	19.8 min
2	ADD, SUBTRACT, MULTIPLY	—	#2	9.7 min
3	DIVIDE, 422 on zero	code	#3	12.7 min
4	Operand limits (BR-1)	code	#4	14.8 min
5	OpenAPI docs	plan + code	#5	12.9 min
5a	Docs off under prod	ponytail	#6	14.0 min

Wall-clock from the “Propose” prompt to the PR merging, including human decisions and CI. Docs + backlog via grill-me: 9.3 min.

Where the minutes went



Minutes, wall-clock from the session's event log. Item 5's review covers the plan and the code; 5a's propose includes a clarifying question.

What it cost: sessions, tokens, dollars

1 session
one Claude Code conversation, no subagents

42 prompts
most of them one-click answers

272 / 379
model responses / tool calls

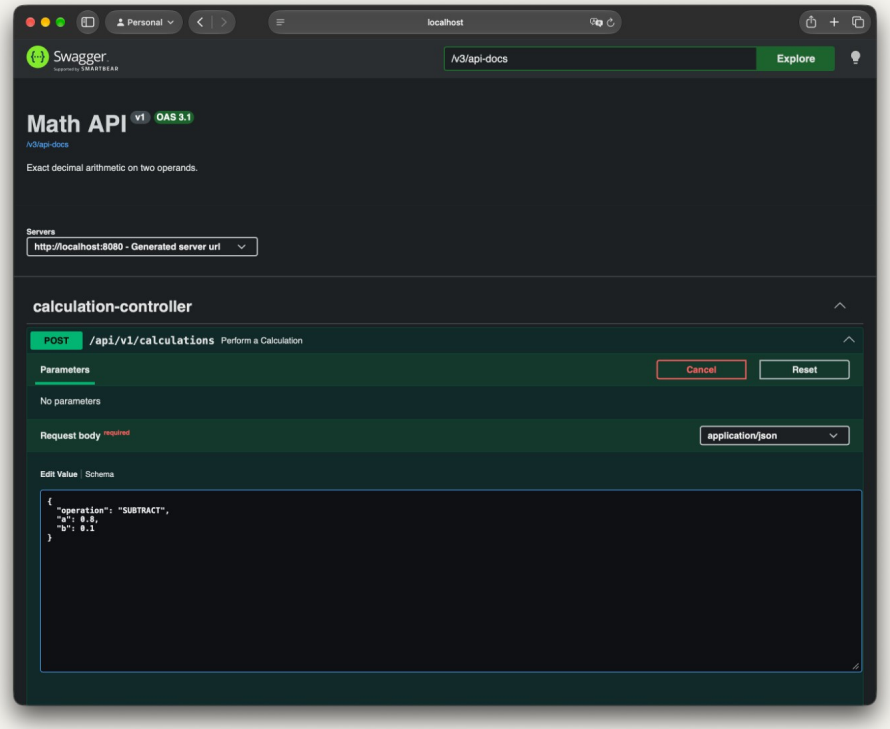
$25.67 at API list prices, Claude Opus 5.5

Tokens	Count	Cost
Cache reads	71.7 M	$14.34
Cache writes (1 h)	0.80 M	$6.36
Output	249 K	$4.97
Uncached input	642	$0.00

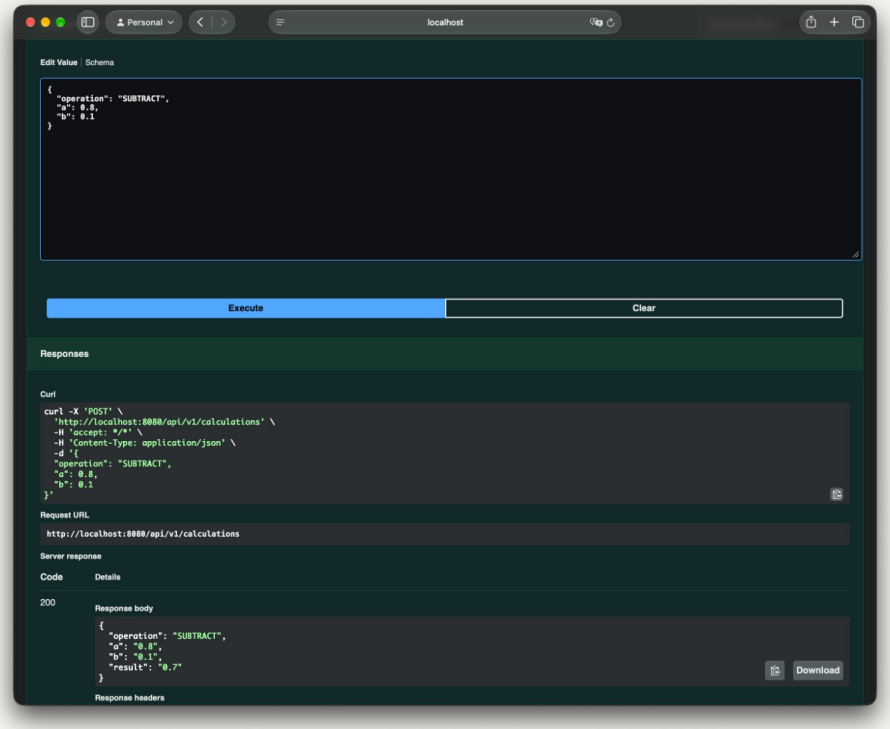
98.6% of tokens were cache reads. Without caching, the same work would have cost about $295.

From the session transcript, 12:37 to 15:41. Per MTok: $4 input, $20 output, $0.20 cache read, $8 1-hour cache write. Making this deck: another $4.56.

The running API, documented by itself



Swagger UI generated at /swagger-ui.html, OAS 3.1



0.8 − 0.1 returns **"0.7"**, exactly

What v1 does

Exact arithmetic

BigDecimal throughout; 0.1 + 0.2 = "0.3"; numbers returned as strings

Divide

34 significant digits, HALF_EVEN; any zero divisor → 422

Problem Details

Every error is RFC 9457 with a stable type; no internals leak

Input limits

≤ 100 digits, scale ±1000, 16 KB body; 1e999999999 rejected in 35 ms

OpenAPI 3.1

Operations generated from the enum; switched off under prod

Clean architecture

Plain-Java domain enforced by ArchUnit; Spotless and CI on every PR

Java 25 · Spring Boot 4.1.2 · Gradle 9.8 · springdoc 3.2 · 404 lines of production code

What made it work

Decide before you build

24 questions up front meant no design arguments mid-change

One prompt per change

The backlog’s ready prompts made each item start with one click

Review before ship

4 of 5 reviews found a real gap, each fixed with a test

Docs as part of done

Config rules put a doc task in every change, so docs never lagged

And the human stayed in charge: every scope call, review and merge was a one-word answer.

Next

Backlog items 6–11: power and modulo, unary operations, discovery, client precision, history, auth

Turn on branch protection so CI must pass before a merge

Deploy with **SPRING_PROFILES_ACTIVE=prod**

```
/opsx:propose — the prompt for item 6 is already in docs/BACKLOG.md
```

strength2 writes the whole loop into CLAUDE.md

“Run each step yourself, without waiting to be asked. Stop only where a step needs the user’s decision.”

1. **Propose** — strict validation passes
2. **Review the plan** — ponytail-review on the artifacts
3. **Fix the plan** — every point, validates again
4. **Apply** — tasks done; tests, types, lint, e2e, Sonar pass
5. **Review the code** — ponytail-review on the diff
6. **Fix the code** — every point, checks re-run
7. **Archive** — specs synced, change archived
8. **Retrospective** — caught, surprises, shortcuts, lessons
9. **Branch and PR** — a code commit, then an archive commit
10. **Merge on green** — once CI passes, then pull main

Every step has a “done when”. Both reviews are mandatory, because both have caught real bugs. Before each new milestone, /ponytail:ponytail-audit runs on the whole repo.

32 of our 42 prompts just said “next step”



The nudges	Times
“Propose item N”	6
“Run /opsx:apply”	6
“Review first”	5
“Fix, then ship”	6
Archive, push, “Check and merge”	9

With the loop in CLAUDE.md, each item is **one prompt** — “do backlog item 6” — and the agent pauses only for the **5 real decisions**.

42 → ~16

6 kickoffs + 5 decisions + 5 requests

Prompts from this session’s event log, 12:37–15:41. Decisions: four grill-me answers and “block the webjar under prod”.

What the strength2 loop would add to ours

Practice	math-api today	With the CLAUDE.md loop
Plan review	1 of 6 changes	Every change, before apply
Code review	4 of 6, when asked	Every change, fix every point
Retrospective	Rebuilt by hand for this deck	retrospective.md in each change
Shortcuts	One, found by grep	ponytail-debt ledger per change
Commits	One squashed commit	Code commit + archive commit
Milestones	No audit	ponytail-audit before each one

The deck's "What the reviews caught" and "Where the minutes went" slides would come straight from the retrospectives.

Proposal for math-api

The same loop, adapted to this repo

Keep the existing “keep the docs current” rule. Swap in our checks: ./gradlew build runs the tests, ArchUnit and Spotless.

Then the whole prompt for the next item is **“do backlog item 6”**.

```
## Change workflow
Run each step yourself; stop only for a decision.
1. /opsx:propose — validate --strict passes
2. ponytail-review the plan, fix every point
3. /opsx:apply — ./gradlew build passes
4. ponytail-review the diff, fix every point
5. /opsx:archive — specs synced, docs updated
6. retrospective.md in the archived change
7. feat/<change> branch, PR via the pr skill
8. squash-merge when CI is green, pull main
## Milestone close
ponytail-audit before item 6 starts v2
```